

Research Proposal:

Does Quantization Noise Act as Implicit Regularization in Low-Data Fine-Tuning of Large Language Models?

Armaan Rehman Shah (1009641309)

Burhanuddin Dalal (1010290365)

STAD68H3

February 13, 2026

Submission Requirements (Mandatory). You must submit **both**:

- A compiled PDF of this proposal (`.pdf`)
- The LaTeX source used to generate it (`.tex`) **plus any required supporting files** (e.g., `.bib`, figures, style files) so the document compiles.

Submissions missing either the `.pdf` or the `.tex` (and required supporting files) are considered **incomplete**.

1 Topic (Required)

- **Problem statement:** We investigate whether **quantization of frozen base model weights** in parameter-efficient fine-tuning (e.g., QLoRA-style setups) acts as an *implicit regularizer* that improves downstream generalization in low-data adaptation. Concretely, we study supervised fine-tuning of causal LLMs using LoRA adapters while varying base-model precision (FP16, 8-bit, 4-bit). We ask: does quantization-induced approximation error reduce overfitting (variance) in small-sample regimes, and under what conditions does it instead harm performance?
- **Motivation:** Parameter-efficient fine-tuning plus quantization are widely used in practice for resource-limited adaptation. QLoRA-style recipes enable single-GPU adaptation of large models, but prior work focuses on feasibility and aggregate performance. A principled study of quantization as an implicit regularizer will (i) clarify when quantization can improve generalization beyond efficiency, (ii) guide practitioners on precision choices in low-data workflows, and (iii) offer empirical counterexamples that motivate future theoretical analysis.
- **Scope:** **In-scope:** empirical characterization of generalization vs data size (100–5k examples), precision (FP16/8-bit/4-bit), and LoRA rank on 2–7B LLaMA-family models under supervised instruction-tuning (e.g., Alpaca-style). Analysis of failure modes (numeracy, rare tokens) and statistical interpretation (bias-variance heuristics). Reproducible code and plots.
Out-of-scope: full fine-tuning of all weights, training models from scratch, RLHF, large-scale multi-GPU experiments (13B+), or formal proofs of generalization bounds for quantized networks (may be future work).

Optional (recommended):

We adopt a controlled experimental protocol using a single base architecture (LLaMA-family 2–7B). We freeze and quantize the base model (FP16, 8-bit PTQ, 4-bit PTQ) and train LoRA adapters at several ranks. We subsample Alpaca instruction data at multiple sizes (100, 500, 1k, 5k) with a fixed held-out test set. Outcomes are compared via validation/test loss, generalization gap, and variance across seeds. We augment this with a concise linear-model analysis to interpret quantization as additive noise and its effect on bias/variance.

2 Summary of Selected Paper(s) (Required)

List the primary paper(s) you will build upon. For each paper, include a full citation in .bib file and briefly summarize them here.

Paper 1

Hu et al. (2021). “Low-Rank Adaptation of Large Language Models (LoRA)”. [1]

Link: <https://arxiv.org/abs/2106.09685>

Key points and relevance:

- Introduces LoRA: freeze pretrained weights and inject trainable low-rank updates $\Delta W = BA$ with $r \ll \min(d_{in}, d_{out})$.
- Provides a parameter-efficient adaptation mechanism widely used in practice.
- Relevance: LoRA isolates adaptation to a low-dimensional subspace, enabling causal attribution of any observed generalization differences to quantization of the frozen base rather than to differing adaptation capacity.

Paper 2

Dettmers et al. (2023). “QLoRA: Efficient Finetuning of Quantized LLMs”. [2]

Link: <https://arxiv.org/abs/2305.14314>

Key points and relevance:

- Proposes quantizing the frozen base model to 4-bit (NF4) while training LoRA adapters in full precision; provides engineering details for single-GPU fine-tuning.
- Empirically demonstrates feasibility and competitive instruction-following performance.
- Relevance: defines the experimental configuration we adopt (frozen quantized base + full-precision LoRA) and highlights the gap: little analysis exists about quantization’s *statistical* effects on generalization.

Paper 3

AskariHemmat et al. (2024). “Quantization and generalization behavior in large models”. [3]

Link: <https://arxiv.org/abs/2404.11769>

Key points and relevance:

- Recent analyses exploring connections between quantization, optimization dynamics, and generalization. (Preprint; used to position our empirical contribution.)

- **Relevance:** informs design of ablations (different PTQ algorithms) and motivates our focus on whether quantization acts like noise-based regularizers.

3 Goal / Target Outcome (Required)

State clearly what you will produce and how success will be measured.

Deliverables

- **Primary deliverable(s):** An empirical study (reproduction + targeted extension) that characterizes when quantization of frozen base weights improves or harms generalization in low-data LoRA adaptation. This is framed as an interpretation/extension of QLoRA rather than a reproduction alone.
- **Artifacts:** Artifacts include a github repo, which includes reproducible code and a public code repository (training + eval scripts), Jupyter notebooks for plotting, trained LoRA adapter checkpoints (small), .pt files for important trained models, evaluation scripts, reproducible experiment logs, and a written report (final paper-style document).

Success Criteria

- **Metrics / evaluation signals:** Token-level cross-entropy loss (NLL) on validation/test sets, perplexity, generalization gap (train vs validation), variance across random seeds, and task-specific accuracy when applicable.
- **Baselines to compare against:** FP16 base + LoRA (baseline), 8-bit PTQ + LoRA, 4-bit PTQ + LoRA; optionally LoRA at multiple ranks and small full fine-tuning runs if compute allows.
- **Claims/hypotheses to test:**
 1. H1: In very low-data regimes (100–500 samples), quantized frozen base (4-bit or 8-bit) leads to smaller validation/test error than FP16 (i.e., acts as implicit regularizer).
 2. H2: The beneficial effect of quantization diminishes as training data increases and may reverse at moderate-to-large dataset sizes.
 3. H3: The effect interacts with LoRA rank: lower ranks amplify the regularizing effect of quantization; higher ranks reduce it.
 4. H4 (counterexample): For numeracy- or rare-token-critical tasks, quantization harms performance even at low data due to structural bias introduced by low precision.

4 Feasibility Plan (Required, Detailed)

This section must evaluate feasibility and explicitly discuss: **(1) repos/codebases**, **(2) datasets**, **(3) playgrounds/tools**, **(4) compute/resources**. Include concrete details and a fallback plan.

4.1 Repositories / Codebases (Required if relevant)

- **Candidate repo(s):**

- QLoRA reference & examples (Dettmers): <https://github.com/artidoro/qlora> (or referenced forks)
- bitsandbytes (quantization kernels): <https://github.com/bitsandbytes-foundation/bitsandbytes>
- Hugging Face transformers + accelerate: <https://github.com/huggingface/transformers>
- GPTQ and AWQ repositories (for PTQ ablations): e.g., GPTQ repo <https://github.com/IST-DASLab/gptq>, AWQ repo references.
- PEFT (LoRA tooling): <https://github.com/huggingface/peft>
- **What you will use it for:** model loading, quantization, LoRA adapter training, evaluation harnesses, and scripts for running controlled ablations and logging.
- **Feasibility assessment:**
 - Install/run status: these repos are actively used in the community and have step-by-step instructions; bitsandbytes and PEFT have pip-installable packages. I have reviewed README and examples and consider them reproducible on a local GPU or cloud instance.
 - Documentation quality: good for transformers/PEFT/bitsandbytes; PTQ repos vary but have usage examples.
 - Activity/licensing: open-source, permissive licenses (MIT/Apache) in general; check specific repos for license details before redistribution.
 - Expected modifications: small wrappers to standardize training loops, fixed-seed reproducibility wrappers, and logging to ensure consistent train/val/test splits across runs.

4.2 Datasets (Required if relevant)

- **Dataset(s):** Primary: **Alpaca Dataset (tatsu-lab/alpaca)** [4] instruction-tuning dataset on Hugging Face: <https://huggingface.co/datasets/tatsu-lab/alpaca>. Optional sanity-check datasets: SST-2 (GLUE) for classification experiments; BoolQ for binary QA.
- **Size/format/access:** Alpaca $\approx$ 52k examples (JSON/JSONL fields: instruction/input/output/text). Publicly accessible via Hugging Face ‘datasets’ library.
- **Preprocessing plan:** Normalize prompts into a canonical instruction $\rightarrow$ response format; tokenize with the target model tokenizer (matching base model). Create one fixed **held-out test set** (10–20% of full dataset) kept constant across all regimes. For each regime, sample training set sizes (100, 500, 1k, 5k) from remaining data and allocate 20% of each training set for validation (or a small fixed validation set for very small N).
- **Ethical/usage constraints:** Alpaca was generated by a teacher model (not human gold), standard citation required. No PII is expected; standard care with model outputs (toxicity) will be followed; results are empirical and do not require human subjects for annotation.

4.3 Playgrounds / Tools (Required if relevant)

- **Framework(s):** PyTorch (primary), Hugging Face Transformers, PEFT, bitsandbytes, GPTQ/AWQ toolkits, ‘datasets’ library, Weights & Biases (or TensorBoard) for logging.
- **Environments/playgrounds:** Local Jupyter notebooks for prototyping; remote GPU instances (cloud or lab server) for experiments.

- **How each tool will be used:** Transformers+PEFT for loading models and applying LoRA; bitsandbytes/GPTQ for quantization; HF datasets for data splits; logging tools for experiment tracking and reproducibility; small scripts to run runs with fixed seeds and to collect metrics.

4.4 Compute / Resources Needed (Required)

- **Hardware:** One consumer GPU with limited VRAM (RTX 3050 Ti laptop GPU, 4–6 GB) is sufficient for 3B models and for 7B models under aggressive 4-bit PTQ + LoRA with small batch sizes. As a fallback, Google Colab (T4/L4 GPUs) will be used for 7B experiments.
- **Estimated runtime:** Each LoRA fine-tuning run (per precision x rank x data size) is expected to take on the order of minutes to a few hours depending on model and batch size. A conservative estimate: **30–120 minutes** per run for 7B model with small datasets. Expecting 30–60 runs total for core ablation grid (precision x ranks x data sizes x 3 seeds).
- **Storage needs:** Dataset (few hundred MB), model checkpoints (LoRA adapters small: tens of MB per checkpoint), quantized base model caches (several GB depending on model). Total safe budget: 50–200 GB.
- **Where compute will come from:** primary plan: local workstation / personal GPU; fallback: cloud (colab Pro+ or GCP/AWS).
- **Fallback plan if compute is limited:** reduce to 3B model, reduce number of seeds, restrict to 2 data sizes (100 and 1k), or run fewer LoRA ranks. Even the minimal setup (3B, FP16 vs 4-bit, N=100/1000) can validate main hypotheses.

5 Preliminary Results (Required)

Preliminary theoretical results:

5.1 Overview

AskariHemmat et al. [3] argue that if the quantized weights w^q can be expressed as $w^q = w + \varepsilon$ with bounded (e.g., uniform) quantization noise ε , then quantization can reduce the *sharpness* of the attained minima. This suggests a regularization-like effect: by smoothing the loss landscape, quantization may improve robustness and mitigate overfitting, particularly in low-data regimes. Motivated by these results, we hypothesize that quantizing the frozen base weights in LoRA fine-tuning will reduce overfitting for LLaMA-family models when only a small number of instruction-tuning examples are available.

5.2 Formal problem setup

We study supervised fine-tuning of an autoregressive language model where the model parameters are separated into a frozen pretrained base W_0 and a low-rank adapter (LoRA) parameterization ΔW . Following the LoRA parametrization [1], for a single linear projection we write

$$\Delta W = BA, \quad B \in \mathbb{R}^{d \times r}, \quad A \in \mathbb{R}^{r \times k},$$

with r the LoRA rank (typically $r \ll d, k$). The model’s output for an input x is

$$\hat{y} = f(x; W_0 + \Delta W).$$

We fine-tune only $\Theta = \{A, B\}$ under the standard autoregressive negative log-likelihood (tokenwise cross-entropy) loss:

$$\mathcal{L}(\Theta) = - \sum_{(x,y) \in \mathcal{D}} \sum_t \log p_{\Theta}(y_t \mid x, y_{<t}).$$

5.3 Quantization: algebraic model and first-order expansion

Quantization of the frozen base weights is modeled per-layer with a learnable step-size s and b bits as in the QGen formulation (and standard PTQ/QAT methods) [3]:

$$\bar{w} = \text{round}(\text{clip}(w/s, -2^{b-1}, 2^{b-1} - 1)), \quad (1)$$

$$\hat{w} = \bar{w} \cdot s, \quad \Delta := w - \hat{w}. \quad (2)$$

Thus quantization induces an error Δ (the “quantization noise”). For the network output $\hat{y} = f(x, w)$ a first-order Taylor expansion in Δ around w yields (componentwise)

$$f(x, w + \Delta) \approx f(x, w) + \nabla_w f(x, w)^\top \Delta.$$

Using mean-square or NLL objectives and taking expectation over Δ with $\mathbb{E}[\Delta] = 0$ gives the key identity (QGen, equation (7) and surrounding derivation):

$$\tilde{\mathcal{L}} = \mathcal{L} + \mathbb{E}_{x,y,\Delta} [\|\Delta^\top \nabla_w \hat{y}\|_2^2] = \mathcal{L} + R(\Delta),$$

so quantization adds an *additional* non-negative term $R(\Delta)$ that depends on the quantization scale and local gradients; this term acts as a regularizer whose magnitude grows with the quantization bin width (i.e., lower bit precision increases $R(\Delta)$). This derivation and discussion is given formally in AskariHemmat et al. (QGen). [3]

5.4 Interpretation as implicit regularization (bias–variance heuristic)

From a statistical perspective, the expected test error decomposes (heuristically) into bias and variance components. The additive regularizer $R(\Delta)$ behaves like a capacity penalty: for small n (low-data) the variance component dominates, and any mechanism that reduces variance at the cost of small bias (including explicit penalties or stochastic perturbations) can improve expected test error. Quantization reduces the effective representational degrees of freedom of the *frozen* portion of the model and introduces a structured perturbation that (under plausible independence approximations) reduces variance of the learned LoRA adapters. QGen formalizes this intuition and supports it with an approximate bound that depends on the scale of quantization noise and gradient norms (see the PAC-Bayes / sharpness-style discussion in Section 4 of QGen). [3]

5.5 Approximate generalization bound (sharpness viewpoint)

Following the sharpness-based PAC-Bayes style bounds used in QGen and related works, one can bound the population loss $L_D(w)$ by a worst-case neighborhood term plus a complexity penalty:

$$L_D(w) \leq \max_{\|\epsilon\| \leq \rho} L_S(w + \epsilon) + h\left(\frac{\|w\|_2^2}{\rho^2}\right),$$

where $h(\cdot)$ is increasing. If quantization restricts the reachable local neighborhood by effectively increasing the minimum attainable local flatness (quantization “pushes” training toward flatter minima when the quantization noise level is moderate) then the $\max_{\|\epsilon\| \leq \rho} L_S(w + \epsilon)$ term can be reduced; QGen shows approximately that quantization with bounded noise can reduce the empirical sharpness and thus improve the RHS of the bound in some regimes. See QGen for a careful derivation and empirical verification on vision architectures. [3]

5.6 Interaction with low-rank adaptation (LoRA)

Write the total weight as $W = W_0 + BA$. The frozen, quantized base contributes $\widehat{W}_0 = W_0 + \Delta$. The model’s output is approximately:

$$\hat{y} \approx f(x, \widehat{W}_0) + \nabla_W f(x, \widehat{W}_0)^\top (BA).$$

Training adapts A, B to fit residuals induced by the target task; the gradients used to update A, B are computed through the *dequantized* activations (QLoRA dequantizes to compute at higher precision). However, the quantization error Δ alters both the residual the adapters must fit and the implicit penalty on adapter solutions: its second-order effect appears as a variance-like penalty on the adapter gradient:

$$\mathbb{E}_\Delta [\|\Delta^\top \nabla_W f\|^2] = \text{tr}(\text{Cov}(\Delta) G^\top G),$$

where G denotes the Jacobian w.r.t. W . When adapters are low-rank this penalty biases solutions toward smaller-norm BA that are less sensitive to frozen-weight perturbations — in other words, quantization helps enforce a *stable low-rank solution* in low-data regimes. LoRA’s empirical finding that very small ranks often suffice [1] supports the plausibility of this mechanism in practice. All numeric values in the following tables are directly quoted from the cited papers and are not reproduced by us.

5.7 Empirical table (QGen): generalization gaps

The QGen paper reports generalization-gap summaries for several architectures and quantization levels. These table values, taken from the paper, illustrate the non-monotone but often beneficial effect of moderate quantization on generalization gap (smaller gap is better).

Table 1: Selected results from QGen. “Generalization” denotes TrainLoss - TestLoss; numbers from QGen Table 1. (Source: AskariHemmat et al., 2024). [3]

Model (dataset)	Precision	Train Acc	Test Acc	Train Loss	Test Loss	Gen. gap
NiN (CIFAR-10)	FP32	97.61	88.05	0.103	0.405	0.302
NiN (CIFAR-10)	Int8	97.50	88.01	0.106	0.407	0.301
NiN (CIFAR-10)	Int4	96.90	87.70	0.125	0.413	0.288
NiN (CIFAR-10)	Int2	93.40	86.11	0.222	0.446	0.224
DeiT-T (ImageNet)	FP32	73.75	71.38	1.38	2.48	1.10
DeiT-T (ImageNet)	Int4	74.71	72.31	1.08	2.07	0.99

5.8 Empirical table (LoRA): rank sweep

LoRA reports that low ranks empirically perform well on adaptation benchmarks. Selected rows from LoRA’s Table 18 (GPT-2, E2E NLG Challenge) are shown below to show how validation loss / BLEU vary with rank: (These are directly quoted from LoRA’s supplemental tables and show non-monotone dependency on rank in some metrics.) [1]

5.9 Synthesis and formal claim (what we can try to prove empirically/analytically)

Claim (informal). Under the first-order quantization model (1)–(2), and assuming (i) the quantization noise Δ is zero-mean and has bounded covariance proportional to the squared quantization

Table 2: Selected LoRA rank results (excerpt from LoRA Table 18).

Rank r	val loss	BLEU	NIST	METEOR	ROUGE-L	CIDEr
1	1.23	68.72	8.72	0.4565	0.7052	2.4329
2	1.21	69.17	8.74	0.4590	0.7052	2.4639
4	1.18	70.38	8.84	0.4689	0.7186	2.5349
8	1.17	69.57	8.75	0.4636	0.7196	2.5196
16	1.16	69.61	8.75	0.4629	0.7177	2.4985

bin width, and (ii) the LoRA parameter Jacobian has bounded Frobenius norm, then quantization contributes an additive regularizer $R(\Delta)$ whose magnitude grows with bin width; consequently, for sufficiently small n (training examples) there may exist a bit-depth b^* such that quantized frozen bases (with LoRA adapters) achieve lower expected test error than the FP16 baseline. The claim matches the conclusions and derivations in AskariHemmat et al. (QGen) [3] and is consistent with the LoRA rank observations that adaptation capacity is small in many tasks.

5.10 Figures

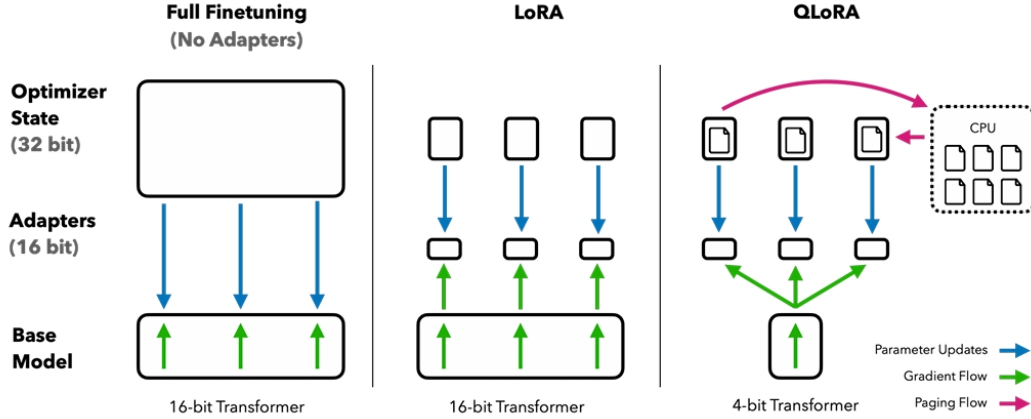


Figure 1: QLoRA memory / method schematic (source: Dettmers et al., QLoRA [2]). This illustrates the QLoRA finetuning pipeline, in which the base model is frozen and quantized to 4-bit precision, while low-rank adapters are trained using standard backpropagation.

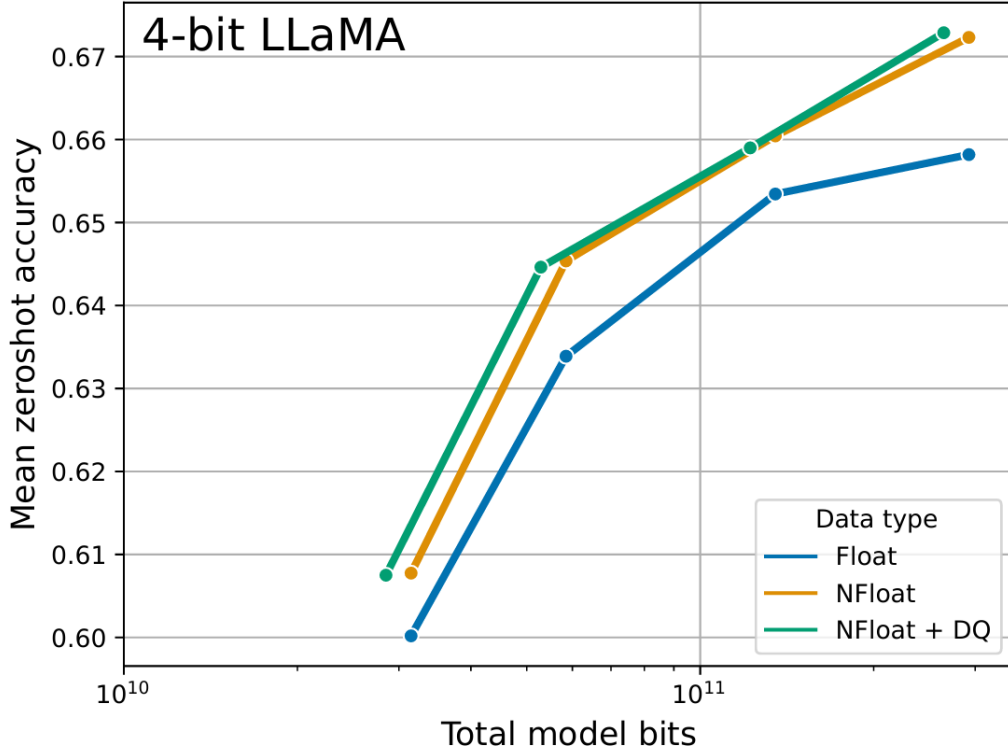


Figure 2: NF4 vs FP4/Int4 comparison (source: Dettmers et al., QLoRA [2]) This figure analyzes NormalFloat-4 (NF4) quantization and shows that it minimizes quantization error under Gaussian weight assumptions, providing theoretical justification for its use in low-bit finetuning. Export from 2305.14314.pdf.

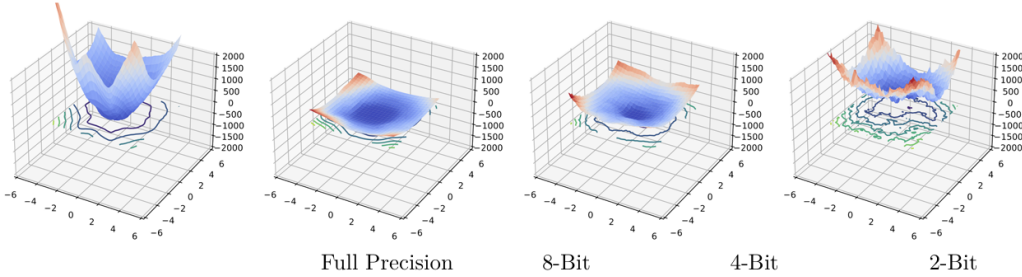


Figure 3: Illustration of loss-landscape flatness for quantized vs FP models (source: AskariHemmat et al., QGen [3]). This figure suggests that quantization can bias optimization toward flatter regions of the loss landscape

5.11 Notes on limitations of the theory

- The Taylor (first-order) expansion treats Δ as small; for extremely low bit depth (e.g., 2-bit) higher-order terms and saturation effects may become important and the simple additive-regularizer view must be revisited. QGen empirically shows non-monotone behavior in such extremes (Int2 cases).

- The sharpness-based generalization bound is sufficient for intuition but not tight; we use it as a heuristic to connect quantization-induced noise to flatter minima (this is the same route QGen uses).

6 Detailed Plan (Required)

Provide a step-by-step plan with milestones. A week-by-week plan is recommended.

1. Milestone 1 (Week 6): Setup & Literature

Expected output: final reading notes, reproducible environment (conda/pip), verify model loading, and run a single LoRA FP16 small-run (3B or 7B depending on hardware) on a tiny subset to verify pipeline.

2. Milestone 2 (Week 7): Baseline experiments

Expected output: Implemented data-splitting (fixed test), run FP16 LoRA baseline for N=100, 500, 1k. Collect train/val curves and confirm overfitting at small N.

3. Milestone 3 (Week 8): Quantization ablation

Expected output: Implement PTQ pipeline (GPTQ/AWQ or bitsandbytes) for 8-bit and 4-bit frozen base models; perform LoRA training and evaluate for each precision at N=100,500.

4. Milestone 4 (Week 9): LoRA rank and seed sweeps

Expected output: Run LoRA rank sweep (e.g., $r = 4, 16, 64$) and 3 random seeds for each major cell to estimate variance. Produce comparative plots of validation error vs data size and precision.

5. Milestone 5 (Week 10): Failure mode and counterexamples

Expected output: Identify tasks where quantization consistently harms (numeracy or rare-token examples), run targeted experiments, and document failure analyses. Optional: small additional dataset (SST-2 or BoolQ) sanity check.

6. Milestone 6 (Week 11): Final experiments & write-up

Expected output: Finalized experiments, polished figures, reproducible code release, and submission-quality report. Prepare short presentation for class/defense.

7 Potential Pitfalls and Mitigations (Required)

List key risks and what you will do if they occur.

- **Pitfall 1:** PTQ tools incompatible with chosen base model or produce unstable quantized checkpoints.
Mitigation: Start with widely-supported LLaMA derivatives and use multiple PTQ implementations (bitsandbytes, GPTQ, AWQ). If necessary, restrict study to 8-bit which is more stable, or to a 3B model.
- **Pitfall 2:** Insufficient compute/time to run full ablation grid.
Mitigation: Prioritize the essential grid (FP16 vs 4-bit x N=100,500,1k x r=16), run fewer seeds, and use 3B models as fallback. Emphasize quality of diagnosis over breadth.

- **Pitfall 3:** No observed regularization effect (negative result).
Mitigation: Negative results are scientifically informative; prepare thorough ablations (LoRA rank, data size, task type) and analyze why quantization does not help (e.g., structured bias, numerical degradation). Frame this as a central contribution.

8 Expected Workload (Required)

Estimate your workload and break it down by activity.

Weekly time estimate

- Total hours/week: **8–12**

Time breakdown

- Reading / paper digestion: 2–3 hrs/week
- Implementation / engineering: 3–4 hrs/week
- Experiments / debugging: 2–3 hrs/week (spikes during ablation weeks)
- Writing / documentation: 1–2 hrs/week (increases near final weeks)

Heaviest components

- **Engineering to obtain stable quantized checkpoints** and ensuring reproducibility (PTQ + model + tokenizer compatibility). This may require adapting community PTQ tools.
- **Running seed-sweep ablations** for statistical confidence—compute time is the primary limiting dependency.

References

- [1] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.
- [2] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized llms. *arXiv preprint arXiv:2305.14314*, 2023.
- [3] MohammadHossein AskariHemmat, Ahmadreza Jeddi, Reyhane Askari Hemmat, Ivan Lazarevich, Alexander Hoffman, Sudhakar Sah, Ehsan Saboori, Yvon Savaria, and Jean-Pierre David. Quantization and generalization behavior in large models. *arXiv preprint arXiv:2404.11769*, 2024.
- [4] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. Stanford alpaca: An instruction-following llama model. https://github.com/tatsu-lab/stanford_alpaca, 2023.